RFID-Based Smart Shopping Basket

Automatic Item Detection, Billing, QR Payment, Cloud Telemetry and AI Anomaly Detection

Project Proposal and Implementation Roadmap

Core skills demonstrated:
STM32 BSP from scratch | Low-level drivers | RFID | SPI/I2C/UART | FreeRTOS | CMake | Conan | MQTT/HTTP | Backend | QR payment workflow | AI anomaly detection

Prepared for: Embedded Systems Portfolio / Product Prototype
Suggested duration: 4-5 months for a strong prototype; 8+ months for a retail-like version

1. Executive Summary

This project proposes an RFID-based smart shopping basket that automatically detects items as soon as they are placed in the basket, updates the cart total, sends the cart data to a backend, and generates a QR code/payment link for the shopper to pay from a mobile phone. The project is a strong end-to-end embedded systems project because the core functionality depends on hardware interfacing, real-time firmware, wireless communication, cloud/backend integration, and product-style application logic.

The first version should be a table-top prototype using STM32, an RFID reader module, RFID tags/cards, an ESP32 Wi-Fi module, and a simple backend. Later versions can move toward UHF RFID, multi-tag reading, payment gateway integration, inventory synchronization, and theft/anomaly detection.

Aspect	Project Coverage
Embedded firmware	STM32 boot, BSP, drivers, interrupts, timers, state machines
Hardware protocols	SPI for RFID, UART for Wi-Fi, I2C for display/EEPROM, GPIO for buttons/buzzer
Application logic	Cart management, duplicate prevention, removal detection, billing engine
Wireless/cloud	Wi-Fi via ESP32, MQTT/HTTP to backend
Payment flow	Backend generates QR code or payment link for the mobile phone
AI extension	Suspicious basket behavior, tag reliability, theft/anomaly detection, shopping patterns

2. Problem Statement and Purpose

Retail checkout is often slow because every item must be manually scanned at the counter. This creates queues, increases staff dependency, and gives a poor customer experience. A smart shopping basket can reduce checkout time by automatically identifying items while the shopper is still shopping.

Purpose of the project:

- Automatically detect RFID-tagged items when they are placed in the basket.
- Maintain a live cart total locally and on the backend.
- Generate a QR code/payment link for quick mobile payment.
- Demonstrate end-to-end embedded product thinking rather than only a sensor demo.
- Create a project suitable for GitHub, LinkedIn, interviews, and embedded systems portfolio building.

3. System Behavior

RFID-tagged item placed in basket
↓
RFID reader detects tag ID
↓
STM32 reads tag over SPI/UART/I2C depending on reader
↓
Firmware maps tag ID to product and price
↓
Cart total is updated
↓
Cart data is sent to backend over Wi-Fi
↓
Backend generates QR/payment session
↓
Shopper scans QR on mobile phone and pays
↓
Payment success is reflected on backend and optionally on basket display

4. High-Level Architecture

RFID-tagged Products
↓
RFID Reader Module
↓
STM32 Microcontroller
↓
Local Basket Logic
↓
Wi-Fi / BLE / Ethernet
↓
Backend Server
↓
QR Code / Payment Link
↓
Shopper Mobile Phone

The embedded device should work even if the cloud connection is temporarily unavailable. The local firmware should maintain the cart state and retry cloud upload once connectivity is restored.

5. Recommended Hardware

Recommended main board: STM32 NUCLEO-H743ZI2. It is powerful enough for serious BSP, driver, RTOS, networking, and application architecture learning. STM32F407 Discovery can also be used for an early proof-of-concept, but the H743 Nucleo gives more expansion flexibility and headroom.

Block	Recommended Part	Interface	Purpose

RFID Smart Shopping Basket Project Roadmap

Main MCU board	STM32 NUCLEO-H743ZI2 / H743ZI	On-chip peripherals	Main embedded controller
Beginner RFID reader	MFRC522 or PN532	SPI/I2C/UART	Prototype item scanning
Retail-like RFID reader	UHF RFID reader module	UART/USB/SPI depending on module	Longer range, multiple tag reading
Wi-Fi	ESP32/ESP8266 AT module	UART	Cloud/backend connectivity
Display	OLED 0.96 inch / LCD	I2C/SPI	Show cart total and status
Buzzer/LED	Generic	GPIO/PWM	Feedback and alerts
Button	Checkout/reset button	GPIO interrupt	Start checkout or reset session
Storage	EEPROM or SPI Flash	I2C/SPI	Product DB cache, offline logs
Power	USB power initially; battery later	Power subsystem	Portable basket prototype

6. RFID Reader Choice

Option	Best For	Advantages	Limitations
MFRC522	Beginner demo	Very cheap, SPI-based, easy to find	Very short range; poor for multiple items
PN532	Better NFC/HF prototype	Supports I2C/SPI/UART; more flexible	Still short range; not true retail basket behavior
UHF RFID reader	Realistic basket/cart version	Longer range, multiple tag reading, closer to retail	More expensive; antenna tuning and false reads are harder

Recommended approach: start with MFRC522 or PN532 to learn the full embedded-to-cloud workflow, then upgrade to UHF RFID once the application architecture is stable.

7. Why This Is a Strong Embedded Project

Project Feature	Embedded Concept Demonstrated
RFID reader integration	Peripheral driver development and protocol handling
STM32 firmware	BSP, clock, startup, linker script, interrupts, timers
Basket detection logic	Real-time state machine and event handling
Display/buzzer/buttons	GPIO, PWM, I2C/SPI
Wi-Fi module	UART driver, AT command parser, retries and error handling

Cart management	Application architecture and data structures
QR/payment workflow	Embedded-to-backend system integration
Removal detection	Filtering, timeout logic, false-read handling
Cloud sync	MQTT/HTTP telemetry and backend APIs
AI/anomaly detection	Pattern recognition based on scan/cart behavior

8. BSP and Low-Level Driver Scope

The project should be implemented in layers so that it resembles a real product firmware rather than a single main.c demo.

```
/firmware
  /startup
    startup_stm32h743xx.s
    linker_script.ld

  /bsp
    bsp_clock.c
    bsp_gpio.c
    bsp_uart.c
    bsp_spi.c
    bsp_i2c.c
    bsp_timer.c
    bsp_exti.c

  /drivers
    drv_rfid.c
    drv_esp32_wifi.c
    drv_oled_display.c
    drv_buzzer.c
    drv_storage.c

  /middleware
    ring_buffer.c
    product_db.c
    basket_manager.c
    json_builder.c
    mqtt_client.c
    qr_payload_builder.c

  /app
    app_scan_manager.c
    app_billing.c
    app_cloud.c
    app_payment.c
    app_main.c
```

```
/tests
  test_basket_manager.c
  test_product_db.c
  test_json_builder.c
  test_fault_detector.c

CMakeLists.txt
conanfile.py
```

Driver/BSP Module	Learning Outcome
Startup file	Reset handler, vector table, initial stack pointer
Linker script	Flash/RAM placement, .data, .bss, heap/stack
Clock BSP	System clock, peripheral clock enable, timing
GPIO driver	LEDs, buttons, buzzer, chip-select pins
SPI driver	RFID reader and optional display/storage
I2C driver	OLED/EEPROM if selected
UART driver	ESP32 AT communication and debug logs
Timer driver	Periodic RFID scan, timeouts, debounce
EXTI driver	Checkout/reset button interrupt
Ring buffer	UART receive buffer and event buffering

9. Application Logic

Core firmware logic:

```
System boot
  ↓
Initialize BSP and drivers
  ↓
Load product database/cache
  ↓
Start basket session
  ↓
Scan RFID tags periodically
  ↓
For each detected tag:
    - check if known product
    - prevent duplicate billing
    - add or refresh item state
    - update total
    - display cart status
    - send update to backend
  ↓
If tag not seen for N scan cycles:
    - mark as temporarily missing
    - after timeout, mark as removed
    - update total
```

```
  ↓
On checkout button:
    - request payment QR/session from backend
    - display QR/link status
    - wait for payment confirmation
```

10. Important Challenge: Item Removal Detection

Adding an item is easier than detecting item removal. RFID reads can be missed because of tag orientation, shielding, basket position, or reader range. Therefore, the firmware should not immediately remove an item just because the tag is missed once.

```
If tag is detected:
  item_state = CONFIRMED_IN_BASKET
  missed_count = 0

If tag is not detected in current scan:
  missed_count++

If missed_count >= TEMP_MISSING_THRESHOLD:
  item_state = MISSING_TEMPORARILY

If missed_count >= REMOVED_THRESHOLD:
  item_state = REMOVED
  update_cart_total()
```

Item State	Meaning
UNKNOWN	Tag has not been mapped or confirmed
DETECTED	Tag appeared in scan
CONFIRMED_IN_BASKET	Item is accepted as present and billed
MISSING_TEMPORARILY	Tag not seen for a few cycles; may be orientation issue
REMOVED	Tag absent long enough; item removed from cart

11. State Machines

Basket-level state machine:

```
IDLE → BASKET_ACTIVE → CART_UPDATED → PAYMENT_READY → PAYMENT_DONE
          ↓
      ERROR / CLOUD_DISCONNECTED / RFID_FAULT
```

Item-level state machine:

```
UNKNOWN → DETECTED → CONFIRMED_IN_BASKET → MISSING_TEMPORARILY → REMOVED
```

12. Backend, Cloud, and QR Payment Flow

For the prototype, real payment integration is not mandatory. Use a mock payment page first. Later, integrate UPI/Razorpay/Stripe/payment gateway links.

STM32 sends cart data to backend
↓
Backend creates checkout session
↓
Backend generates QR code/payment URL
↓
Shopper scans QR from display/mobile page
↓
Mock payment page shows total and Pay button
↓
Backend marks payment complete
↓
STM32 receives payment status through polling or MQTT message

Prototype Level	Payment Implementation
Level 1	QR opens a local/web page that shows cart total and a Mark as Paid button
Level 2	QR opens a payment link generated by backend
Level 3	Integrate real payment provider and payment status callback

13. Sample Payloads

Cart update payload from STM32 to backend:

```
{
  "basket_id": "basket_001",
  "session_id": "S2026_0001",
  "items": [
    {"tag_id": "04A1B2C3D4", "sku": "MILK_500", "name": "Milk 500 ml", "qty": 1, "price": 35},
    {"tag_id": "0488CC129A", "sku": "BREAD_001", "name": "Bread", "qty": 1, "price": 45}
  ],
  "total": 80,
  "currency": "INR",
  "state": "CART_UPDATED"
}
```

Payment response payload from backend to STM32:

```
{
  "session_id": "S2026_0001",
  "payment_status": "PAID",
  "amount": 80,
```

```
  "transaction_id": "TXN123456"
}
```

14. FreeRTOS Task Design

Task	Responsibility
ScanTask	Periodically scan RFID reader and generate tag events
BasketTask	Maintain cart state, duplicate prevention, removal detection
DisplayTask	Update OLED/LCD with item and total
CloudTask	Send cart updates and receive payment status
WiFiTask	Manage ESP32 connection and AT command responses
ButtonTask	Handle checkout/reset button events
LoggerTask	Store important events and errors

Use queues between tasks. For example, ScanTask sends TAG_DETECTED events to BasketTask. BasketTask sends CART_UPDATED events to DisplayTask and CloudTask.

15. CMake and Conan Plan

Start with simple IDE or Makefile only for the earliest bring-up if needed, but migrate to CMake once LED/UART/SPI are stable. Introduce Conan after CMake, mainly for host-side tests and reusable modules.

Tool	Use in Project
CMake	Build firmware, define ARM GCC toolchain, generate ELF/BIN/MAP, organize modules
Conan	Manage host-side unit test dependencies, package reusable utilities, support cross-build experiments
Python	Backend, QR generation, AI/anomaly detection, data analysis
GitHub Actions optional	Build host tests and static checks

```
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake
cmake --build build
arm-none-eabi-size build/firmware.elf
```

16. AI and Pattern Detection Ideas

AI should be added after the basic embedded-to-cloud flow is stable. Do not start with deep learning. Start with rule-based and statistical anomaly detection, then move to Python-based models.

AI Feature	Purpose
Suspicious basket behavior	Detect repeated add/remove cycles or unusual checkout behavior
Tag read reliability analysis	Detect bad tag placement or failing reader/antenna
Theft/anomaly detection	Detect mismatch between detected items, checkout state, and payment state
Shopping pattern analysis	Find items commonly bought together
Inventory insight	Estimate popular products and movement patterns

Useful first AI/anomaly rules:

- Same item appears and disappears repeatedly within a short time.
- Basket reaches exit/payment state without successful payment.
- RFID reader misses too many scans compared to normal baseline.
- High-value item removed shortly before checkout.
- Cart total changes abnormally often in a short session.

17. Implementation Timeline

Phase	Work	Deadline	Main Output
1	Board bring-up + GPIO/UART/SPI	Week 3	LED blink, UART logs, SPI basic communication
2	RFID driver + tag reading	Week 6	Read RFID tag IDs reliably
3	Product database + billing engine	Week 8	Tag ID mapped to product and total
4	Display + buzzer + buttons	Week 10	Cart feedback on local device
5	Wi-Fi communication	Week 13	ESP32 connects and sends data
6	Backend + QR generation	Week 16	QR/payment page generated for cart total
7	Payment simulation + final demo	Week 18	Complete end-to-end demo
8	AI/anomaly add-on	Week 22	Pattern detection and alerting

Expected duration:

- Basic working prototype: 3 to 4 months.
- Good portfolio version: around 5 months.
- Retail-like version using UHF RFID and robust payment/inventory integration: 8+ months.

18. Phase-Wise Deliverables

Phase 1 - Board Bring-Up

- Startup file and linker script
- Clock initialization
- GPIO driver
- UART debug print
- SPI skeleton
- Document reset flow and memory map

Phase 2 - RFID Driver

- Initialize RFID reader
- Read tag ID
- Handle RFID errors
- Avoid duplicate repeated reads
- Unit test tag parser on host where possible

Phase 3 - Billing Engine

- Product database
- Cart add/remove APIs
- Total calculation
- Duplicate billing prevention
- Session management

Phase 4 - User Feedback

- OLED/LCD display
- Buzzer/LED indications
- Checkout button
- Reset session button
- Error display states

Phase 5 - Wireless Communication

- ESP32 AT driver
- Wi-Fi connection state machine
- MQTT/HTTP publish
- Retry and timeout handling
- Offline queue

Phase 6 - Backend and QR

- Backend API
- Cart database
- QR code generation
- Payment simulation page

- Payment status callback/polling

Phase 7 - Final Demo

- End-to-end video demo
- README with setup steps
- Architecture diagram
- Known limitations
- Future improvements

Phase 8 - AI Add-on

- Collect session logs
- Define anomaly labels
- Python analysis
- Simple anomaly model
- Dashboard/alert output

19. Risks and Practical Limitations

Risk	Why It Matters	Mitigation
HF RFID short range	MFRC522/PN532 may not detect all basket items	Use as prototype only; upgrade to UHF for real basket
Missed reads	Tags may disappear due to orientation or shielding	Use missed-scan thresholds and state machine
Multiple item detection	Cheap readers may not support reliable anti-collision	Prototype one item at a time; later UHF module
Payment integration complexity	Real gateways need compliance and credentials	Start with mock payment page
Power design	Portable basket needs battery design	Begin with USB power; add battery later
Scope creep	Project can become too large	Build in versions: MVP, portfolio version, retail-like version

20. MVP Definition

The minimum impressive version should do the following:

1. STM32 boots using your BSP and prints logs over UART.
2. RFID reader detects at least 5 different tagged items.
3. Each tag is mapped to a product name and price.
4. Cart total updates correctly with duplicate prevention.
5. OLED/display or UART shows current cart total.
6. ESP32 sends cart JSON to a backend.

7. Backend generates a QR code/payment page.
8. Mock payment success is shown on the device or console.

21. Version Roadmap

Version	Description	Target
V1	Table-top prototype with MFRC522/PN532 and mock payment	Learning and proof of concept
V2	Better basket behavior with display, Wi-Fi, removal detection, local storage	Portfolio-ready demo
V3	UHF RFID, multiple tag detection, payment gateway, inventory backend	Retail-like prototype
V4	AI anomaly detection, anti-theft logic, analytics dashboard	Advanced product showcase

22. Resume and GitHub Positioning

Suggested project title:

> STM32-Based RFID Smart Shopping Basket with Automatic Billing, QR Payment Workflow, Cloud Telemetry, and AI-Based Anomaly Detection

Resume bullet examples:

- Designed an STM32-based smart shopping basket prototype using RFID-based item detection, automatic billing, and QR payment workflow.
- Implemented custom BSP and low-level drivers for GPIO, SPI, UART, timers, and RFID reader integration.
- Developed cart state machine with duplicate prevention, missed-read filtering, and item removal detection logic.
- Integrated ESP32 Wi-Fi module to publish cart updates to a backend using MQTT/HTTP.
- Built backend workflow to generate QR/payment session and return payment status to the embedded device.
- Added Python-based anomaly detection for suspicious basket behavior and tag-read reliability analysis.

23. Final Verdict

This is a strong, purposeful embedded systems project. It is more impressive than a simple sensor-to-cloud demo because it solves a clear retail problem and requires real embedded design: hardware

interfacing, BSP, low-level drivers, real-time state machines, wireless communication, backend integration, QR payment flow, and optional AI analytics.

The key to completing it successfully is to avoid starting with a supermarket-grade system. Build a table-top MVP first, then improve the RFID range, multiple item detection, payment integration, battery design, and analytics in later versions.

24. Useful References and Learning Links

These references are useful starting points while implementing the project:

- STM32 NUCLEO-H743ZI / H743ZI2 board documentation and examples from STMicroelectronics.
- STM32CubeH7 firmware package examples for clock, GPIO, UART, SPI, I2C, Ethernet, and FreeRTOS reference.
- ARM CMSIS documentation for Cortex-M startup, core registers, and standard device access.
- MFRC522 / PN532 module datasheets and example command sequences.
- ESP32 AT command documentation for Wi-Fi and TCP/MQTT connectivity.
- CMake documentation for cross-compilation toolchain files.
- Conan documentation for C/C++ package management and cross-build profiles.